PROJECT DOCUMENTATION

MICROSERVICES-BASED ORDER PROCESSING SYSTEM

Design, Implementation, API Contracts, Testing and Docker Deployment

Submitted by

Jajula NagaMani

B.Tech — Computer Science and Engineering
Chalapathi Institute of Engineering and Technology
Academic Year 2026–2027

A project report prepared for academic and technical reference

DECLARATION

I, Jajula NagaMani, declare that this report titled “Microservices-Based Order Processing System” has been prepared to document the design and development work associated with the project. The report is intended to present the system purpose, architectural decisions, service boundaries, request flows, deployment approach, and verification plan in a structured manner.

Descriptions of observed environment behavior are based on the Docker Compose build and startup logs available during preparation of this report. Where a feature or verification step has not been established by source code or test output, it is described as a design proposal, recommended practice, or pending validation rather than as a completed result.

The diagrams and illustrative request examples are included to explain the intended architecture. They should be reconciled with the final source code, configuration, database entities, and API responses before the report is submitted for formal assessment.

I understand that the final submitted copy must accurately represent the actual implementation and that any references, reused assets, or third-party materials should be acknowledged according to institutional guidelines.

ACKNOWLEDGEMENT

I express my sincere gratitude to the faculty members of the Computer Science and Engineering department at Chalapathi Institute of Engineering and Technology for providing an academic environment in which software engineering concepts can be explored through practical implementation.

I am thankful to my project guide and department for their guidance, feedback, and encouragement. Working on a microservices-oriented system provides an opportunity to connect classroom learning with practical concerns such as API design, independent service responsibilities, persistence, inter-service communication, containerization, and deployment troubleshooting.

I also acknowledge the maintainers of the open-source technologies used in this project, including the Spring ecosystem, MySQL, Maven, JUnit, Mockito, and Docker. Their documentation and community knowledge are important resources for understanding and applying these tools responsibly.

Finally, I thank my peers and family for their encouragement. This project is an ongoing learning exercise, and the documentation will continue to improve as service-level tests, end-to-end request validation, and operational checks are completed.

Acknowledgement note: Edit this page to match the actual people and support you wish to acknowledge.

ABSTRACT

The Microservices-Based Order Processing System demonstrates how an order workflow can be divided into independently deployable services. The design contains four business services: Customer Service, Product Service, Order Service, and Notification Service. Each service owns a focused responsibility and uses a separate MySQL database so that its data can be managed within a clear service boundary.

Customer Service provides customer lookup through GET /api/customers/{id}. Product Service exposes a stock-reduction operation through PUT /api/products/{id}/reduce-stock. Order Service accepts order creation requests through POST /api/orders and coordinates the workflow. Notification Service accepts notification requests through POST /api/notifications. REST over HTTP is used for synchronous inter-service communication in the current design.

The application stack includes Java, Spring Boot, Spring Data JPA, MySQL, Maven, and Docker Compose. The declared project technology also includes JUnit 5 and Mockito for automated testing. Container build logs show that service images were built and that all four Spring Boot services started on ports 8081–8084 with successful MySQL connection-pool initialization. The container build command used Maven with tests skipped, so these startup logs must not be interpreted as proof that automated tests or the complete order workflow passed.

This report documents the requirements, architecture, UML diagrams, API design, data ownership, error handling, testing approach, deployment operations, limitations, and future enhancements. End-to-end business behavior remains subject to explicit API and integration testing.

Keywords: Microservices, Spring Boot, REST API, Spring Data JPA, MySQL, Docker Compose, order processing, service boundaries.

EXECUTIVE SUMMARY

The project addresses a common backend engineering problem: coordinating customer validation, stock changes, order persistence, and customer-facing notification without placing every responsibility in one large application. The chosen microservices approach divides the domain into four services and deploys them as separate application containers.

Customer Service is responsible for customer records and lookup. Product Service owns product and inventory data. Order Service owns order records and coordinates calls to other services. Notification Service owns notification requests and delivery status records. Separate databases reinforce data ownership: a service should access its own persistence directly and use a documented API when it needs information from another service.

Docker Compose is used to define the runtime environment and start service containers together with four MySQL containers. The observed build uses Maven 3.9.11 with an Eclipse Temurin 25 build image and a Temurin 25 Java runtime image; the logs identify MySQL 8.0.46. The service logs show application startup on ports 8081, 8082, 8083, and 8084, with database connection pools reaching the completed state.

The primary engineering challenge is not simply to start four services; it is to ensure that their APIs work together correctly when requests succeed or fail. Inventory reduction and order persistence cross service boundaries and therefore cannot be assumed to share one database transaction. A production-ready design should explicitly manage compensation, duplicate requests, timeouts, retries, and notification failure. These concerns are covered as design recommendations and validation items throughout this report.

- Main deliverable: four-service backend architecture with separate data ownership.
- Observed milestone: Docker images built and the four Spring Boot services started.
- Pending proof: automated test results, API contract validation, and full order-flow verification.

TABLE OF CONTENTS

Front Matter

Certificate 2

Declaration 3

Acknowledgement 4

Abstract 5

Executive Summary 6

List of Figures, Tables and Abbreviations 8

Main Chapters

1. Introduction 9

2. Background and Domain Context 10

3. Problem Statement and Motivation 11

4. Objectives, Scope and Assumptions 12

5. Existing and Proposed Systems 13

6. Feasibility Study 14

7. Requirements Specification 15

8. Use Cases and Actors 16

9. Architecture Overview 17

10. System Architecture Diagram 18

11. Microservice Design Principles 19

12–15. Service Specifications 20–23

16–22. APIs, Communication, Data, Workflow, Errors, Security and Quality .. 24–30

23–25. UML Diagrams 31–33

26–27. Implementation and Development Setup 34–35

28–30. Docker, Testing and Test Cases 36–38

31–35. Results, Runbook, Client Integration, Limitations and Future Work 39–43

36. Conclusion and Viva Preparation 44

Appendices and References 45

Navigation: Page numbers are planned for this 45-page layout. If you edit text or change fonts in Word, repaginate and refresh the table of contents before submission.

LIST OF FIGURES, TABLES AND ABBREVIATIONS

Figures

Figure 1. System Architecture Overview — page 18

Figure 2. UML Use Case Diagram — page 31

Figure 3. UML Component Diagram — page 32

Figure 4. Order Placement Sequence Diagram — page 33

Tables

Table 1. Service endpoints and ports — page 17

Table 2. Project objectives and success criteria — page 12

Table 3. Functional and non-functional requirements — page 15

Table 4. API contract summary — page 24

Table 5. Logical data entities — page 26

Table 6. Verification scenarios — page 38

Abbreviations

Term	Meaning
API	Application Programming Interface
CRUD	Create, Read, Update, Delete
DTO	Data Transfer Object
HTTP	Hypertext Transfer Protocol
JPA	Jakarta Persistence API
REST	Representational State Transfer
SUT	System Under Test
UML	Unified Modeling Language
CI/CD	Continuous Integration / Continuous Delivery
DB	Database

1. INTRODUCTION

Order processing is a core capability in many online and internal business applications. A typical request depends on several types of information: whether the customer exists, whether a product is available, whether the order can be stored, and whether a notification should be created. These responsibilities are related, but they do not have to live in one deployable unit.

A microservices architecture organizes an application around small, independently deployable services aligned with business capabilities. Each service exposes an API and manages its own data. This structure can improve ownership and make it possible to scale or release individual capabilities independently. It also introduces additional engineering challenges: network calls can fail, data is distributed, and a workflow can partially complete.

This project implements a learning-oriented order-processing domain with four services. Customer Service provides customer lookup; Product Service manages inventory updates; Order Service receives and stores orders; and Notification Service records or dispatches notification requests. The services communicate through REST endpoints and are deployed in Docker containers.

The purpose of this report is to explain the intended behavior and design decisions in a form that is useful for academic evaluation, maintenance, troubleshooting, and technical interviews. It treats startup evidence separately from functional correctness. Container startup is an important milestone, but correctness must also be demonstrated through representative API requests, database verification, and automated tests.

- Explain the domain and requirements before discussing implementation.
- Show how service boundaries and APIs support the business workflow.
- Document deployment and a repeatable verification procedure.

2. BACKGROUND AND DOMAIN CONTEXT

The domain contains four key business concepts. A customer represents the person or organization placing an order. A product represents an item that can be ordered and has an inventory quantity. An order records a purchase intention and its associated items, quantities, total, and status. A notification records a communication requested by the system after an order event.

In a single-process application, one service method might access all tables directly. In a microservices design, each capability owns its persistence and provides a stable interface. This difference matters because another service must not depend on internal database tables or ORM entities belonging to a different service.

The current communication style is synchronous REST. The Order Service sends HTTP requests and waits for responses from Customer and Product Service. It also sends a notification request to Notification Service. Synchronous communication is straightforward to understand and debug, but request latency depends on the services called and a downstream outage can affect the calling request.

The project uses Spring Boot to create HTTP services, Spring Data JPA to simplify persistence, MySQL for relational storage, and Docker Compose to define local containers and networking. Maven is used to package services. JUnit 5 and Mockito are designated tools for test automation. Their use in a final submission should be supported by test classes and execution reports rather than inferred from dependency declarations alone.

Domain concept	Responsible service	Primary responsibility
Customer	Customer Service	Identify and return customer details
Product / stock	Product Service	Read products and apply inventory changes
Order	Order Service	Validate request and persist order lifecycle
Notification	Notification Service	Accept a message request and record its status

3. PROBLEM STATEMENT AND MOTIVATION

A business requires a backend that can process customer orders while keeping customer, product, order, and notification responsibilities separate. The backend must verify that the customer is valid, check and update stock, persist the order, and request a notification. As these operations span independent services, the system must handle both successful responses and failure paths.

A monolithic implementation can be easier to begin with but may create tight coupling when unrelated business capabilities share code, data models, and release schedules. A microservices approach offers stronger service boundaries, but a naive distributed workflow can create inconsistency. For example, stock could be reduced while order persistence fails, or an order could be persisted while notification delivery fails.

The motivation is to learn how real backend systems use API contracts, service decomposition, database ownership, containerization, and integration testing. The project is also intended to demonstrate the difference between service availability and business correctness. An application may start and connect to a database yet still expose an incorrect endpoint, reject a valid payload, or mishandle a downstream error.

Core problem to solve

Design an order workflow that coordinates four independent services through well-defined REST APIs and can be started and inspected through Docker Compose. Define the desired behavior for unavailable customers, missing products, insufficient inventory, persistence errors, repeated requests, and notification failures. Verify the behavior through API tests and automated service-level tests.

- Avoid direct cross-service database access.
- Reduce ambiguity in request and response contracts.
- Make failure behavior observable and testable.
- Document setup so another developer can reproduce the runtime.

4. OBJECTIVES, SCOPE AND ASSUMPTIONS

Objectives

ID	Objective	Evidence / success criterion
O1	Separate the four business capabilities	Four separately built Spring Boot services
O2	Use REST for inter-service calls	Documented endpoint and payload contracts
O3	Separate persistence ownership	A dedicated MySQL database per service
O4	Containerize the runtime	Dockerfiles and Compose configuration build and start
O5	Define a testing strategy	JUnit 5 / Mockito tests plus API integration cases
O6	Improve operational clarity	Centralized error shape, readable logs, troubleshooting guide

Scope boundary: These are target objectives. Build and startup are evidenced by logs; the remaining criteria require the stated implementation artifacts and tests to be checked.

5. EXISTING AND PROPOSED SYSTEMS

Typical existing approach

A small application may keep customer, product, order, and notification logic inside one deployment and share one database. This is initially simple: one transaction manager can coordinate local data changes, and there are fewer network calls. Over time, however, the application may become harder to release independently, and teams can develop accidental dependencies on shared tables or modules.

Proposed microservices approach

The proposed system separates the four responsibilities into Customer Service, Product Service, Order Service, and Notification Service. Each service has its own database and communicates through explicit REST endpoints. The Order Service acts as the workflow coordinator for a place-order request. Containerization gives each service a repeatable runtime package.

Comparison

Dimension	Single application	Proposed architecture
Deployment	One unit	Four application services
Data ownership	Often shared schema	Database per service
Inter-service calls	In-process method calls	REST over HTTP
Failure handling	Often local exception flow	Timeouts and partial completion must be managed
Scaling	Scale whole application	Services can be scaled independently
Operational effort	Lower initial complexity	More configuration, logs and network dependencies

Trade-off: Microservices are not automatically faster or simpler. They are useful when separate ownership and deployment boundaries justify the operational overhead.

6. FEASIBILITY STUDY

Technical feasibility

The selected stack is appropriate for a learning project focused on Java backend development. Spring Boot provides embedded HTTP servers and configuration conventions. Spring Data JPA supports relational persistence. MySQL provides familiar transactional storage. Docker Compose can orchestrate several application and database containers on a local machine.

Economic feasibility

The stack is based primarily on open-source tools and can be developed on a personal computer. Local execution avoids mandatory cloud costs. The main cost is development time spent on configuration, testing, and diagnosing interactions across services.

Operational feasibility

A developer can start the stack with a Compose command and review service logs from a single terminal. Separate databases create additional containers and configuration values, so resource use and startup order need to be managed. Health checks and readiness dependencies are recommended so that services do not attempt database operations before databases are ready.

Schedule feasibility

The project can be developed incrementally: first start each service independently, then validate local CRUD operations, then implement REST clients, then test the complete order flow, and finally automate repeatable checks. This sequence reduces the number of simultaneous unknowns.

Conclusion

The project is feasible as an academic demonstration. Production suitability depends on the completion of reliability, security, observability, and testing controls described later in this report.

7. REQUIREMENTS SPECIFICATION

Functional requirements

ID	Requirement	Priority
FR-01	Customer Service shall provide customer lookup by identifier.	Must
FR-02	Product Service shall expose product inventory data.	Must
FR-03	Product Service shall reject stock reduction when quantity is invalid or insufficient.	Must
FR-04	Order Service shall accept an order-creation request.	Must
FR-05	Order Service shall verify the customer before accepting the order.	Must
FR-06	Order Service shall invoke product stock validation / reduction.	Must
FR-07	Order Service shall persist the resulting order record.	Must
FR-08	Order Service shall request notification after the order workflow reaches its defined success point.	Should
FR-09	Failures shall be returned as meaningful HTTP responses.	Must
FR-10	Service and database configuration shall be reproducible using Docker Compose.	Must

Non-functional requirements

Maintainability: keep controllers, services, repositories, entities, and API DTOs in clear layers. Reliability: set connection and request timeouts and define safe retry behavior. Security: validate inputs, protect secrets, restrict management endpoints, and use TLS when deployed outside a trusted local environment. Observability: log request outcomes without exposing credentials or unnecessary personal data. Testability: isolate business logic and test both success and failure paths. Portability: use environment-based configuration so service containers do not depend on hard-coded host paths.

Acceptance reminder: Convert each requirement into a repeatable test or review checklist before final submission.

8. USE CASES AND ACTORS

The primary external actor is the Customer, represented in this learning system by an API client or a future user interface. The actor initiates a place-order request and receives an outcome. The services themselves are internal system components, not human actors.

Primary use cases

Use case	Trigger	Preconditions	Expected outcome
View customer details	GET /api/customers/{id}	Customer ID supplied	Customer details or not-found response
Check product stock	Product lookup or order request	Product ID valid	Current product/availability information
Reduce stock	PUT /api/products/{id}/reduce-stock	Product exists; quantity valid	Inventory reduced once, or clear failure
Place order	POST /api/orders	Request valid; customer and product checks succeed	Order saved and response returned
Request notification	POST /api/notifications	Defined order event exists	Notification accepted or failure reported

Use-case relationships

The place-order use case includes customer verification and product stock processing as required steps. Notification creation is associated with the order workflow but should not be confused with guaranteed delivery. If the notification service is unavailable, the design needs an explicit policy: fail the whole request, mark notification pending, or publish an event for later retry. For a robust business system, persisting the order independently from transient notification delivery is usually preferable, with an outbox or durable event mechanism added as the design matures.

Actor note: The current endpoint-oriented design does not prove that a graphical customer UI exists. Treat a frontend as an API consumer unless its implementation is separately verified.

9. ARCHITECTURE OVERVIEW

The system is organized into four Spring Boot application services. Each service exposes its own HTTP interface and connects to a dedicated MySQL database. The Order Service is the coordinating component for the order workflow. It calls Customer Service to verify the customer, calls Product Service to validate and reduce inventory, persists order information in the Order database, and requests a notification from Notification Service.

The observed local container ports are 8081 for Customer Service, 8082 for Product Service, 8083 for Order Service, and 8084 for Notification Service. Within a Docker Compose network, containers should call each other through Compose service names and container ports, rather than through localhost. The localhost address inside a container points back to that same container.

The project uses four MySQL containers in the supplied Compose environment. This gives each service a database boundary and reduces accidental coupling through shared tables. Database credentials and environment configuration should be injected via environment variables or a secure secret mechanism rather than committed in public source control.

The design shown in this report treats REST calls as synchronous unless the implementation explicitly provides asynchronous messaging. The request flow and actual API names should always be checked against controller mappings and configured client URLs before the report is submitted.

Service	Port	Endpoint highlighted in requirements	Database
Customer	8081	GET /api/customers/{id}	customer_db
Product	8082	PUT /api/products/{id}/reduce-stock	product_db
Order	8083	POST /api/orders	order_db
Notification	8084	POST /api/notifications	notification_db

10. SYSTEM ARCHITECTURE DIAGRAM

Figure 1 summarizes the intended service boundaries and the main order-processing interactions. The diagram is a conceptual architecture view: confirm exact routing, service names, database names, and response arrows against the project configuration and source code.

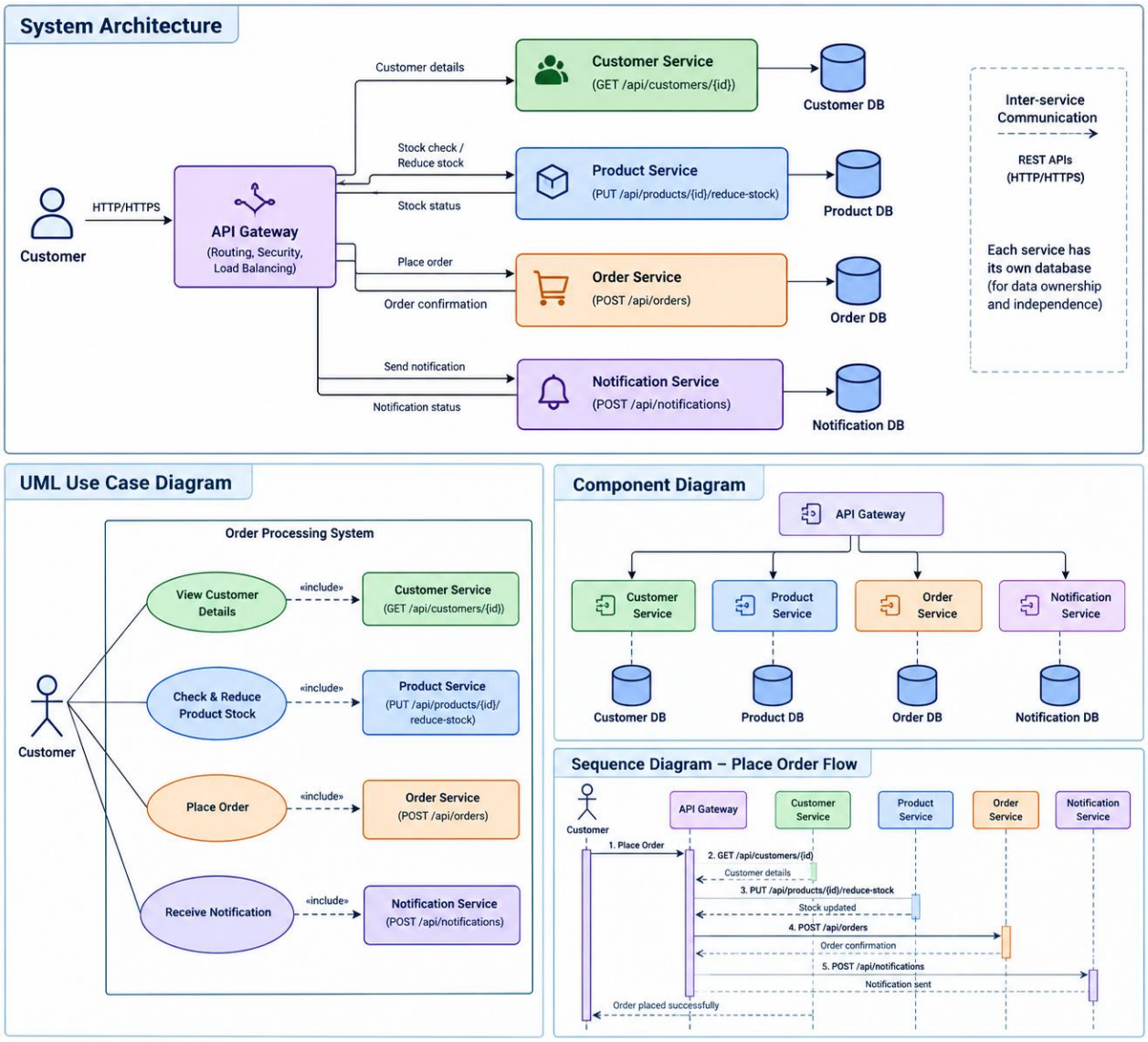


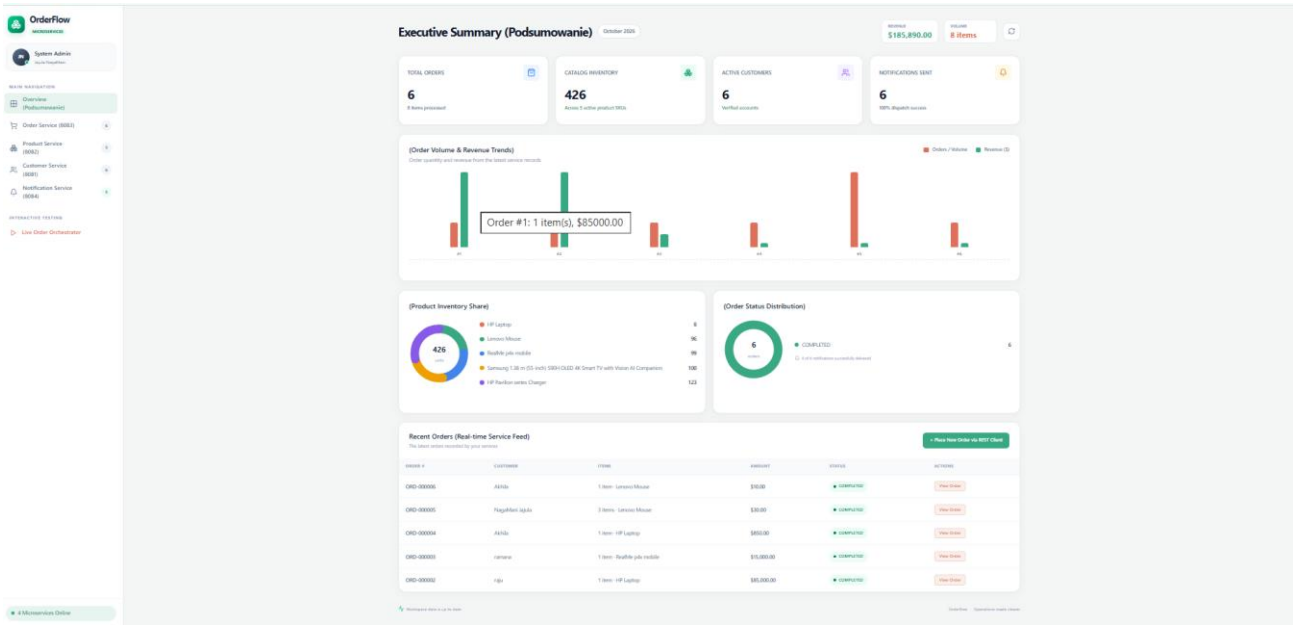
Figure 1. Architecture and UML overview prepared for this project.

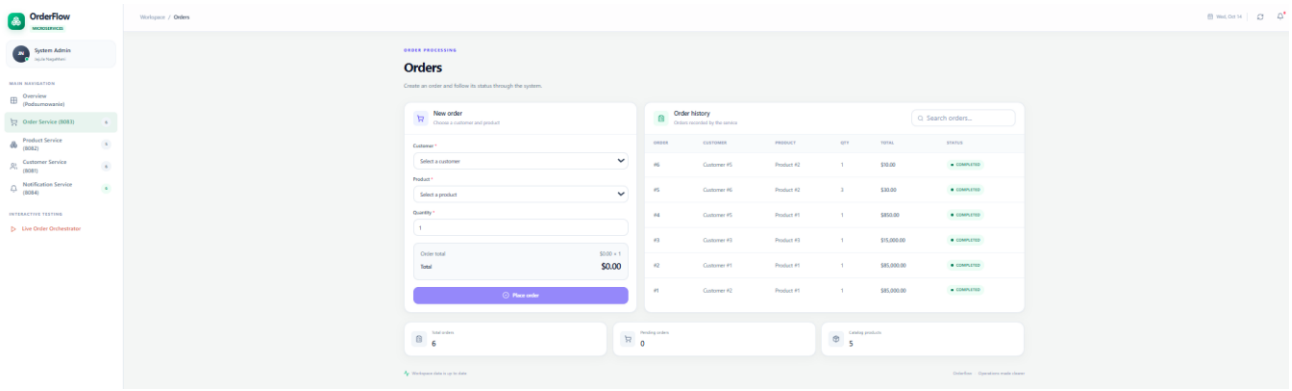
Important design detail: The graphic includes an API Gateway as a conceptual entry point. Add or retain a gateway in the implementation description only if one actually exists in the repository; otherwise treat the client as calling Order Service directly.

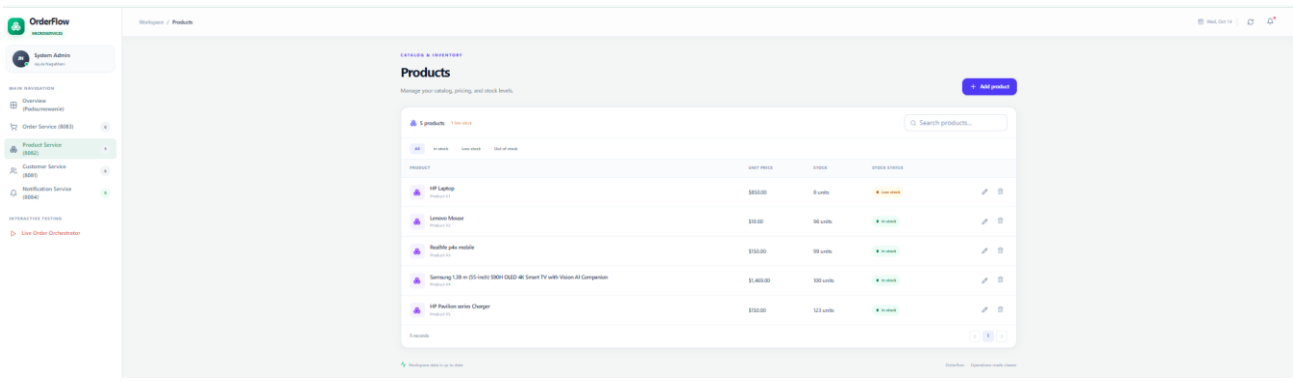
Frontend Dashboard UI & Component Design (React + Tailwind CSS)

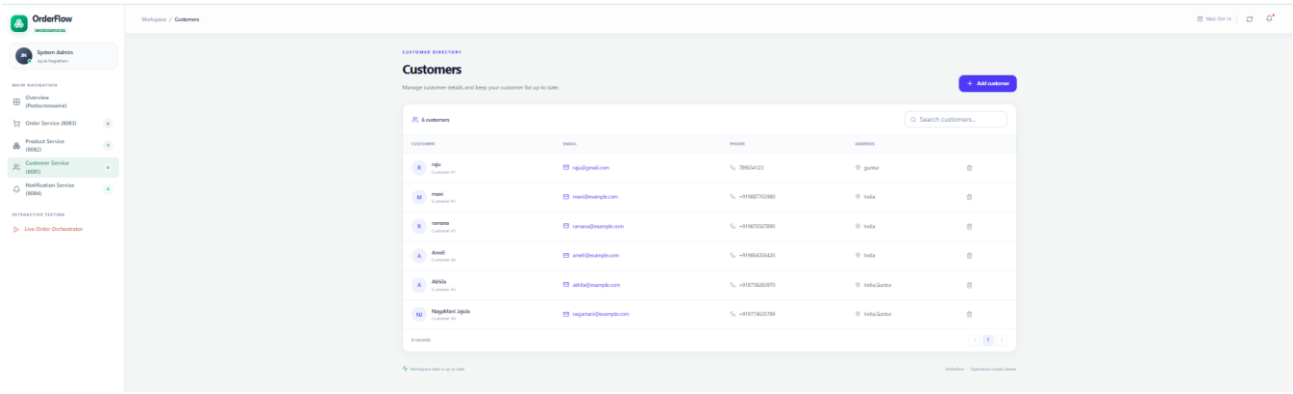
The frontend is built inside an isolated /frontend directory using Vite, React, Tailwind CSS, Lucide React icons, and Axios. It provides five primary views:

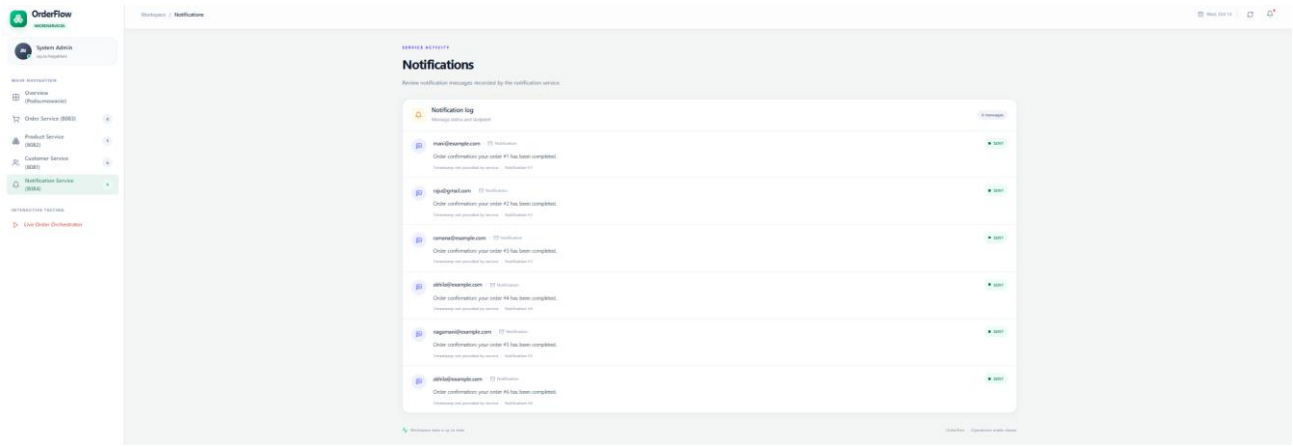
1. **Dashboard / Overview: High-level metrics cards displaying active customers, total products, completed orders, and system activity logs.**
2. **Customer Management: A clean data table with modal forms to add, edit, or delete customer records.**
3. **Product Catalog & Inventory: Grid/table views of product items showing pricing and stock badges, with quick action modals to update stock levels.**
4. **Order Processing Checkout: An intuitive form allowing administrators to select a customer and product, input quantities, view real-time price calculations, and submit orders with instant status feedback.**
5. **Notification Logs: Audit feed displaying dispatched customer confirmation messages, recipient emails, and delivery statuses.**


OrderFlow
Executive Summary (Podsumowanie)
$185,890.00
8 items
6
426
6
6
Order #1: 1 item(s), $85000.00
(Product Inventory Share)
(Order Status Distribution)
Recent Orders (Real-time Service Feed)


OrderFlow
Orders
New order
Order history
$0.00
Place order


OrderFlow
Products
Add product





11. MICROSERVICE DESIGN PRINCIPLES

Single business responsibility

Each service should represent a cohesive capability. Customer Service owns customer data and customer-specific business rules. Product Service owns product and inventory rules. Order Service owns the order lifecycle. Notification Service owns notification requests, delivery state, and notification-specific behavior.

Data ownership

A service should read and write its own database. Another service must not directly query the service database, because doing so couples it to internal schema and makes independent changes difficult. Cross-service information should be retrieved through an API or replicated through a deliberate event-driven design.

Stable contracts

APIs should use explicit DTOs rather than expose persistence entities directly. DTOs allow database implementation details to change without changing the wire contract. Validation annotations and service-level checks should be used for request fields, and response status codes should distinguish invalid input, missing resources, conflicts, and unexpected failures.

Independent deployment

Each service has a Docker build and application configuration. Shared libraries should be kept small and should not reintroduce tightly coupled domain logic. A change to one service should not require direct access to another service’s source code or database schema, except where a documented contract requires coordinated versioning.

Failure is normal

Network calls can time out or return an error. Apply explicit timeouts, avoid unlimited retries, and ensure retries are safe. For inventory and order consistency, prefer a documented compensation or reservation model over assuming that all four services participate in one ACID transaction.

12. CUSTOMER SERVICE SPECIFICATION

Customer Service owns customer records and exposes lookup behavior to other components. The key endpoint in the project requirements is GET /api/customers/{id}. The Order Service uses this endpoint to ensure that the supplied customer identifier refers to an acceptable customer before placing an order.

Responsibilities

- Create, retrieve, update, or deactivate customer data where those operations exist in the implementation.
- Validate identifier format and return an appropriate response when the customer is not found.
- Keep customer persistence and repository access inside this service.
- Return a DTO with only fields needed by callers.
- Log request outcome with a correlation identifier; avoid logging sensitive personal details.

Item	Recommended behavior
Input	Path variable customer ID
Success	200 OK with customer DTO
Missing ID	400 Bad Request if the route/input is malformed
Unknown customer	404 Not Found
Database failure	500 or service-unavailable response with a safe error body

Illustrative response

The exact fields must be aligned with the implemented Customer DTO. Do not expose unnecessary contact or personal information simply because it exists in the database.

```
HTTP/1.1 200 OK
Content-Type: application/json

{
  "id": 101,
  "name": "Example Customer",
  "status": "ACTIVE"
}
```

Implementation check: Verify the controller mapping, DTO properties, not-found behavior, validation rules, and database seed data. The example response is illustrative, not a claim about the current code.

13. PRODUCT SERVICE SPECIFICATION

Product Service owns product details and inventory quantity. The specified stock-reduction endpoint is PUT /api/products/{id}/reduce-stock. It changes state and therefore must validate the requested reduction before writing to the database. It should reject non-positive quantities and ensure that stock never becomes negative.

Inventory invariants

- Product ID must refer to an existing product.
- Reduction quantity must be greater than zero.
- Available stock must be greater than or equal to the requested quantity.
- The inventory update should be atomic to prevent concurrent requests from overselling.
- A failed update must not report success or silently change stock.

Concurrency considerations

A read-check-write sequence can race when two orders attempt to reduce the same stock. For safer implementation, use an atomic conditional update, optimistic locking, or a row-level transaction. For example, an update can include a condition such as stock_quantity >= requested_quantity and treat zero affected rows as insufficient stock or a concurrent modification. The exact mechanism should be consistent with the entity mapping and database strategy.

```
Illustrative request body
{
  "quantity": 2
}

Illustrative success body
{
  "productId": 501,
  "remainingStock": 8,
  "status": "STOCK_UPDATED"
}
```

Implementation check: The requirements provide the route, but the request payload shape, status code, and response fields must be confirmed from the actual controller and DTO.

14. ORDER SERVICE SPECIFICATION

Order Service is the central coordinator for order placement. It accepts POST /api/orders, validates the incoming order DTO, verifies the customer with Customer Service, requests stock reduction from Product Service, persists the order in order_db, and invokes Notification Service according to the chosen success policy.

Suggested processing stages

- Validate required fields, identifiers, quantities, and item count.
- Check that the customer is valid and eligible to order.
- Validate each product and request stock reservation or reduction.
- Calculate totals on the server using trusted product prices; do not trust client-submitted totals.
- Persist the order and its line items with a defined status.
- Trigger notification only after the event point chosen by the design.
- Return an order identifier, status, and a stable response representation.

Transaction boundary

A local database transaction can make the Order Service's own order tables atomic, but it cannot automatically include Product Service's independent database. The endpoint must therefore define what happens if inventory is changed but order persistence fails. For a simple academic version, explicitly document and test compensation (restore stock) or clearly describe the remaining consistency limitation. For a more robust version, use a reservation state and a saga-style workflow.

Important: The name "transactional" should refer to the Order Service's local persistence transaction unless distributed transaction coordination is actually implemented and verified.

15. NOTIFICATION SERVICE SPECIFICATION

Notification Service accepts notification requests through POST /api/notifications. It is responsible for validating notification payloads and recording or dispatching communication according to the supported channels. The presence of this service does not by itself prove that real email, SMS, push, or messaging-provider delivery is configured.

Recommended request fields

Field	Purpose	Validation guidance
eventType	Identifies the business event	Allow only supported values
recipient	Target recipient or reference	Validate format and minimize stored personal data
subject	Short notification heading	Apply length limits
message	Notification content	Apply length limits and output-encoding controls
correlationId	Links logs across services	Optional but strongly recommended
idempotencyKey	Prevents duplicate processing	Recommended for retryable requests

Failure handling

If the service stores a notification request and then calls an external provider, separate the “accepted” state from the “delivered” state. Use a controlled status model such as PENDING, SENT, FAILED, and RETRYING where appropriate. Do not log message bodies or recipient details by default. A retry should use a stable request identifier so that a repeated HTTP call does not create duplicate messages.

Implementation check: Document the actual notification channels, status values, and persistence behavior only after confirming the service code.

16. REST API DESIGN AND CONTRACTS

REST endpoints are the public contracts between services. Resource paths should be consistent, payloads should be expressed through DTOs, and response semantics should be predictable. The four routes below are taken from the project requirements and are the baseline for Postman validation.

Service	Method and path	Purpose	Common success response
Customer	GET /api/customers/{id}	Fetch customer details	200 OK
Product	PUT /api/products/{id}/reduce-stock	Reduce inventory	200 OK or 204 No Content
Order	POST /api/orders	Create order	201 Created or 200 OK
Notification	POST /api/notifications	Submit notification request	201 Created or 202 Accepted

Contract conventions

Use 400 for malformed or invalid input, 404 for a missing resource, 409 for a domain conflict such as insufficient stock or an idempotency-key conflict, and 5xx responses for unexpected server or downstream failures. These are recommendations, not verified current response codes. Return a consistent error body containing a timestamp, status, error code, message suitable for clients, request path, and correlation identifier. Do not return stack traces or database details.

Example order request

This payload is illustrative. Replace fields and nesting with the exact OrderRequest DTO exposed by the implementation.

```
POST /api/orders
Content-Type: application/json

{
  "customerId": 101,
  "items": [
    { "productId": 501, "quantity": 2 }
  ]
}
```

API verification: Check actual request DTOs, validation annotations, status codes, and response JSON before using these examples as final API documentation.

17. INTER-SERVICE COMMUNICATION

Synchronous inter-service communication uses HTTP requests between Spring Boot services. In Docker Compose, service discovery is generally achieved through stable service names on the project network. For example, Order Service should call a configured base URL for Customer Service using the Compose DNS name and the service's internal port. The base URL must not use localhost from inside another container.

Communication pattern

Caller	Target	Request	Expected decision
Order Service	Customer Service	GET /api/customers/{id}	Continue only for a valid customer
Order Service	Product Service	PUT /api/products/{id}/reduce-stock	Continue only when stock operation succeeds
Order Service	Notification Service	POST /api/notifications	Apply documented notification policy

Timeouts and retries

Every outgoing HTTP call should have a connection timeout and a response timeout. Retries should be limited, should apply only to appropriate failures, and should use idempotent operations or an idempotency mechanism. Retrying stock reduction without an idempotency key can reduce stock twice if the first request succeeded but the response was lost.

Circuit breaking and resilience

For a small local project, basic timeouts and clean error mapping are a practical starting point. As the system grows, consider circuit breakers, bulkheads, retry budgets, and metrics. These controls should be configured deliberately rather than added without an understanding of their effect on duplicate requests and transaction consistency.

Correlation

Generate or propagate a request correlation ID across each service call. Include the ID in structured logs so an order request can be traced through customer lookup, stock update, order persistence, and notification handling.

18. DATABASE DESIGN AND DATA OWNERSHIP

The system uses separate MySQL databases to reinforce independent service ownership. The conceptual schema below is logical guidance; entity names, columns, relationships, and constraints must be compared with the source code and the actual database before being represented as implemented facts.

Database	Illustrative entity	Important attributes	Ownership rule
customer_db	Customer	id, name, status, contact fields	Only Customer Service changes customer data
product_db	Product	id, name, unit_price, stock_quantity	Only Product Service changes product and stock data
order_db	Order / OrderItem	order_id, customer_id, status, total, product_id, quantity, unit_price_snapshot	Only Order Service manages order records
notification_db	Notification	id, event_type, recipient_ref, status, created_at	Only Notification Service manages notification records

Referential information across services

Because databases are separated, a customer ID stored on an order is a logical reference, not a cross-database foreign key. The Order Service should validate the customer through Customer Service at the appropriate time. Similarly, order item product identifiers should not imply that the Order Service can directly query product_db. Store a price snapshot on order items when historical totals must remain stable after a product price changes.

Schema management

Use explicit schema migration tooling for non-trivial projects and make schema changes backward compatible when services are deployed independently. Production deployments should avoid relying on automatic destructive schema updates. Database credentials should be unique per service and restricted to that service’s database where practical.

Verification task: Inspect each entity, repository, table name, column type, index, and relationship in the actual implementation.

19. ORDER WORKFLOW AND DATA CONSISTENCY

Order placement spans several independent resources. A single local @Transactional method cannot atomically commit both the order database and the product database. This is one of the defining challenges of a distributed workflow.

Proposed happy path

- Receive and validate the order request.
- Fetch the customer and verify eligibility.
- Request a stock reservation or reduction from Product Service.
- Create and commit the order and its items in order_db.
- Record or dispatch the notification request.
- Return the order identifier and current order status.

Partial-failure examples

Case A: customer lookup fails. The order should not be persisted and stock must not change. Case B: the product does not exist or has insufficient stock. The request should return a clear domain error and no order should be confirmed. Case C: stock is reduced but order persistence fails. The system must compensate by restoring stock or use a reservation that can be released. Case D: order persistence succeeds but notification fails. A robust design keeps the order and marks notification delivery pending rather than falsely reporting that the order never existed.

Saga-style coordination

A saga models a business process as local transactions with compensating actions. An inventory reservation can be released if order creation fails. A notification can be retried independently after the order is accepted. For stronger durability, an outbox record can be committed with the order and processed asynchronously. These are recommended extensions unless explicitly implemented in the current source code.

Consistency rule: The API response should clearly distinguish order creation success from notification delivery success.

20. ERROR HANDLING AND LOGGING

Errors are expected in distributed systems and should be reported consistently. Controllers should not return raw exception messages or database implementation details to callers. Domain errors should have predictable status codes and stable error identifiers so that API clients can react without parsing arbitrary text.

Centralized error response

In Spring Boot, a shared exception-handling component can map validation failures, missing resources, conflicts, downstream timeouts, and unexpected exceptions into a common response DTO. A suitable shape is shown below as a recommendation; use the project's actual exception class names and fields in the implementation.

Logging guidelines

Use structured logs with service name, timestamp, level, operation, request or correlation ID, and outcome. Log enough context to diagnose failures but avoid passwords, tokens, database connection strings, full payment or identity data, and complete notification bodies. Stack traces belong in controlled service logs, not public error responses.

Operational signals

Important events include service startup, database connection failures, rejected validation, downstream timeout, stock conflict, order commit, and notification status changes. Metrics for request rate, latency, error percentage, and dependency health improve diagnosis as the application grows.

```
Illustrative error response
{
  "timestamp": "2026-10-13T17:20:00Z",
  "status": 409,
  "code": "INSUFFICIENT_STOCK",
  "message": "Requested quantity is not available",
  "path": "/api/orders",
  "correlationId": "demo-7f12a8"
}
```

Current evidence: Startup logs prove that the applications reached the started state and their Hikari pools completed initialization. They do not prove that centralized error handling or structured correlation logging is implemented.

21. SECURITY CONSIDERATIONS

The documented project requirements focus on order processing and container deployment. Even in a learning project, security should be considered from the start because services accept identifiers, quantities, and message content over HTTP.

Input validation

Validate required fields, positive quantities, identifier format, string length, and allowed status values at the API boundary. Enforce business invariants again in the service layer; client-side checks are not sufficient. Do not accept client-calculated totals as authoritative.

Secrets and database access

Do not commit live passwords or production credentials into version control. Use environment variables for local development and a managed secrets mechanism for deployed environments. Use separate database accounts per service with the minimum required permissions. Avoid using an administrator account from application containers.

Transport and access control

For local development, plain HTTP may be acceptable on a trusted host. For remote or production environments, terminate TLS at a trusted gateway or configure TLS between services according to the threat model. Define which APIs are public and which are internal. Add authentication and authorization if the system is exposed to real users or networks.

Data minimization

Return only the customer fields needed for order processing and avoid storing duplicate personal information unless necessary. Notification endpoints must validate recipients and reject content that violates business rules. Logs should not become a second store of sensitive user data.

- Rotate any credential that has been committed accidentally.
- Restrict exposed host ports to only those required for local testing.
- Apply dependency updates and review known vulnerabilities before release.

22. RELIABILITY, PERFORMANCE AND SCALABILITY

Availability

The availability of the order workflow depends on its synchronous downstream dependencies. If Customer Service or Product Service is unavailable, the Order Service may be unable to validate a new order. Notification Service should ideally be decoupled from order acceptance so that a transient notification problem does not erase a committed order.

Latency

Each synchronous call adds network time to the request. An order request that calls customer, product, and notification services can accumulate multiple network and database delays. Set timeouts to fit a documented latency budget and avoid unbounded waits. Capture latency at both the caller and callee so the slow dependency can be identified.

Data consistency and concurrency

Use atomic stock operations and avoid duplicate inventory updates. Consider an idempotency key for order creation and stock-reduction requests. For parallel processing, define order state transitions and ensure each transition is legal. Database indexes should support commonly used identifier lookups.

Scaling

Stateless services can generally be replicated behind a load balancer, provided that shared state remains in external databases and idempotency is coordinated. Scaling the application containers also increases database connection demand, so connection pool sizes and database capacity should be considered together. The local Compose setup is a development environment, not evidence of production-scale resilience.

Health checks

Separate process liveness from readiness. Liveness should indicate that the process can continue running; readiness should indicate whether it can safely handle requests. Docker health checks and Compose dependency conditions can improve startup behavior, but an HTTP “started” log alone is not a complete readiness assessment.

23. UML USE CASE DIAGRAM

Figure 2 shows the Customer actor interacting with the principal user-visible actions. The use case diagram emphasizes what the system provides rather than its internal class structure. The API endpoints are annotated beside the associated service blocks to make the connection between functional requirements and service contracts visible.

“Place order” is the primary use case. In its intended workflow, the system checks the customer, checks and reduces product stock, stores an order, and requests a notification. The exact inclusion relationships should reflect the actual behavior: if notification delivery is asynchronous or optional, it should not be modeled as a mandatory prerequisite for the order to exist.

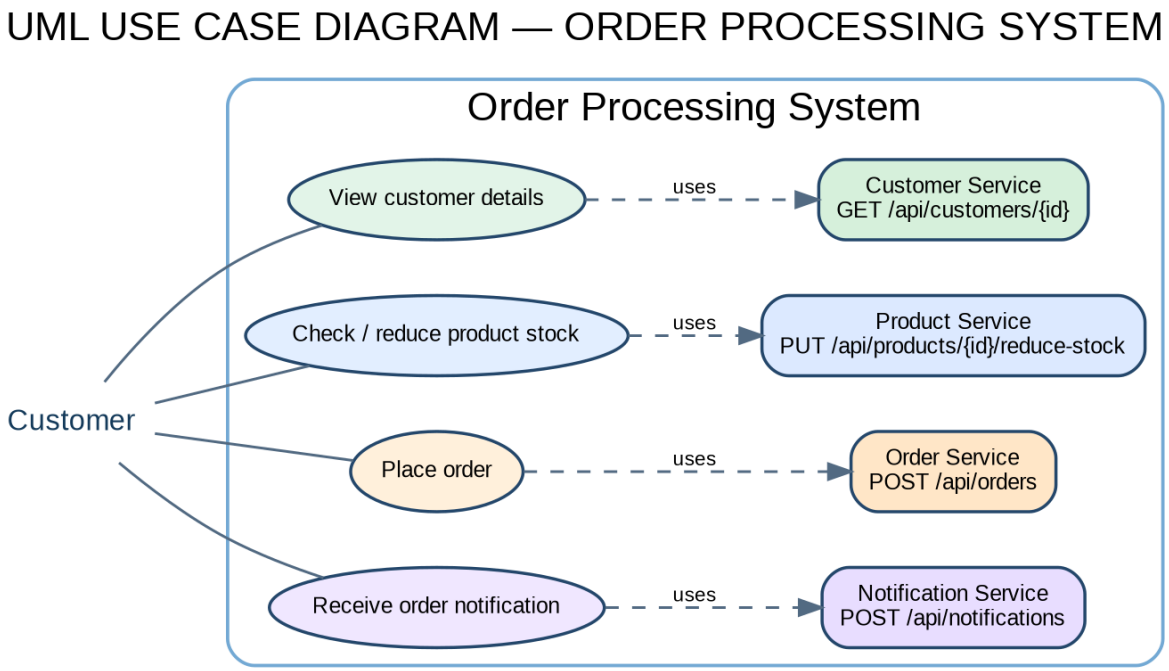


Figure 2. UML-style use case diagram for the order-processing system.

Modeling note: This is a use-case-oriented view with service endpoint annotations for readability. Update it if the implemented scope adds authentication, payment, cancellation, or a customer-facing user interface.

24. UML COMPONENT DIAGRAM

Figure 3 shows the major software components, their REST dependencies, and their corresponding databases. Component diagrams are useful for discussing deployment boundaries and dependency direction. The Order Service coordinates the process but does not own customer or product persistence.

The arrows represent logical dependencies rather than proof that every call has been exercised. The HTTP client configuration, Compose service names, ports, and controller mappings should be inspected to confirm the diagram. A real API Gateway should be shown only if the repository includes and configures one; otherwise the client can call Order Service directly.

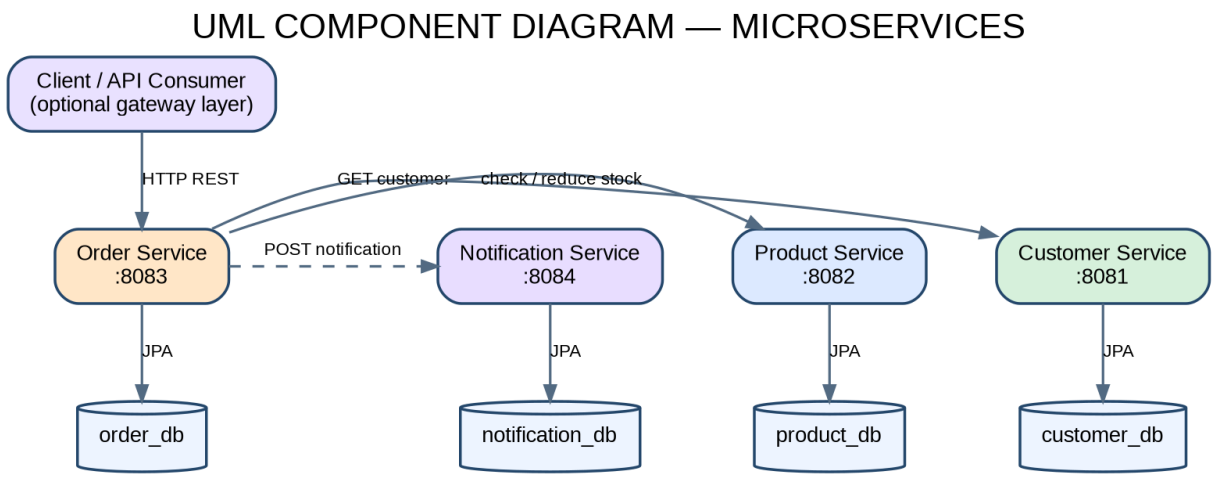


Figure 3. Components, REST dependencies, and database ownership.

Boundary rule: Do not implement a direct order-service SQL connection to customer_db or product_db. Communicate through those services' public APIs.

25. UML SEQUENCE DIAGRAM — PLACE ORDER

Figure 4 illustrates one possible sequence for a successful order request. The client sends POST /api/orders. Order Service queries Customer Service, asks Product Service to update inventory, persists the order in its own database, and requests a notification. Responses flow back to Order Service and then to the client.

The order of inventory update and database persistence is a design choice with consistency consequences. If stock is reduced before the order is committed, a persistence failure can require compensation. If the order is committed before inventory is reserved, the order may need an intermediate PENDING state until stock is confirmed. Document which ordering is actually implemented and include failure tests for it.

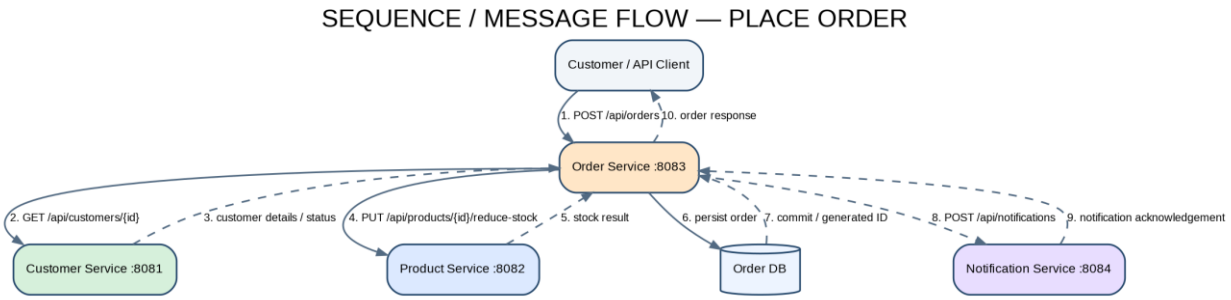


Figure 4. Illustrative message sequence for successful order placement.

Sequence limitation: The diagram is a proposed message flow, not a trace captured from a successful Postman run. Validate it against the source code and runtime logs.

26. IMPLEMENTATION STRUCTURE AND CODE ORGANIZATION

A maintainable Spring Boot service commonly separates web/API handling from business rules and persistence. The exact package structure may differ between the four modules, but responsibility should remain clear.

Layer / package	Responsibility	Design guidance
controller	HTTP mappings, request/response handling	Keep methods thin; delegate business work
dto / request / response	API data contracts	Validate inputs; avoid exposing entity internals
service	Business rules and orchestration	Own workflow decisions and transaction boundary
repository	Database access	Use Spring Data JPA queries and repositories
entity / model	Persistent domain representation	Use correct types, constraints, and relationships
exception / advice	Domain errors and error mapping	Return stable error structures
client / integration	Calls to other services	Centralize URLs, timeouts, and mapping

Suggested package layout

Use this as a target layout only; retain the actual module and package names already established in the project.

```
com.example.orderservice
  controller/
  dto/
  service/
  repository/
  entity/
  client/
  exception/
  config/
  OrderServiceApplication.java
```

Implementation review checklist

Confirm that business rules do not live only in controllers, repositories are injected consistently, DTO validation is active, transactions are placed around local database changes, and external HTTP clients map downstream failures into meaningful domain responses. Avoid circular dependencies between services and avoid sharing persistence entities across service modules.

Alignment requirement: Replace the illustrative package prefix above with the project's exact package names before submitting.

27. DEVELOPMENT ENVIRONMENT AND BUILD PROCESS

The project is developed on a Windows machine using PowerShell and Docker Desktop. The build logs show a Maven-based multi-stage container build using Maven 3.9.11 with Eclipse Temurin 25 for the build stage and Eclipse Temurin 25 JRE for the runtime stage. MySQL 8.0.46 images are shown in the Compose logs.

Typical commands

Build versus test

The captured Docker build log includes a Maven package command with -DskipTests. This means the container build intentionally skipped running automated tests. Image creation and service startup should therefore be recorded as separate from test execution. Run the Maven test lifecycle independently for each service or add a deliberate test stage to continuous integration.

Environment configuration

Place environment-specific values in Compose variables or an ignored local environment file. Check that host-exposed ports do not conflict with other applications. From a container, use Compose DNS names for sibling services and database containers.

```
# From the project directory; use the Compose file path in this repository
docker compose -f ".\microservices-order-system\docker-compose.yml" config
docker compose -f ".\microservices-order-system\docker-compose.yml" up --build

# Run in background after verifying the stack
docker compose -f ".\microservices-order-system\docker-compose.yml" up -d --build

# Check container state
docker compose -f ".\microservices-order-system\docker-compose.yml" ps

# Follow one service
docker compose -f ".\microservices-order-system\docker-compose.yml" logs -f order-service
```

Reproducibility: Before a clean demonstration, verify the Compose file with config, inspect service health, and run the exact documented API requests.

28. DOCKER CONTAINERIZATION AND DEPLOYMENT

Docker packages each Spring Boot service into an image and runs the images as containers. Docker Compose defines the application containers, MySQL containers, network, environment variables, port mappings, and startup dependencies. The supplied project logs show successful image builds for Customer, Product, Order, and Notification services and the creation of a shared Compose network.

Multi-stage build

A multi-stage Dockerfile uses a Maven image to compile and package the application and a smaller Java runtime image to run the resulting JAR. This separates build tooling from runtime. The observed logs show the Maven package phase used -DskipTests, so test validation must be completed separately.

Networking

Services can resolve each other by Compose service name on the internal network. Host port mappings such as 8081:8081 make a service accessible from the developer machine, while service-to-service calls should target the internal service name and listening port. Database connections should use the database service name and database port 3306 within the Compose network.

Startup readiness

Compose startup order does not automatically guarantee that a database is ready to accept connections unless health checks and dependency conditions are configured. Use health checks when appropriate and make applications tolerate transient startup delays. Read the logs for Hikari pool completion and Tomcat startup to confirm initialization milestones.

- Use docker compose ps to review container state.
- Use docker compose logs --tail=100 service-name to inspect recent logs.
- Use docker compose down to stop and remove containers and the default network.
- Avoid docker compose down -v unless deleting named/anonymous volumes and their data is intended.

29. TESTING STRATEGY

Testing should be organized into layers so that defects can be isolated. Unit tests validate business logic without starting a database or making real network calls. Repository tests validate persistence mappings. Controller tests validate routes, payload validation, and HTTP responses. Integration tests validate a service with its database and configuration. End-to-end tests validate the full order workflow across all four services.

JUnit 5 and Mockito

JUnit 5 provides test organization, lifecycle, and assertions. Mockito can replace repositories or HTTP clients with controlled test doubles. A unit test for Order Service can simulate customer-not-found, insufficient-stock, successful stock reduction, and notification timeout without requiring all containers to run. These tests should verify both returned results and important side effects.

Test data isolation

Use deterministic test fixtures and isolated database state. Tests should not depend on manually inserted rows in a developer database. Where repository behavior matters, use a dedicated test database or containerized test database. Ensure tests clean up or roll back state so they can be rerun consistently.

Automation

Run tests as part of Maven's test phase and collect the Surefire reports. The Docker build log currently uses -DskipTests, which is acceptable as a packaging choice only if tests are executed in a separate verified step. Record the exact command, test count, and failures in the final test evidence section.

Level	Primary target	Typical tools
Unit	Service business rules	JUnit 5, Mockito
Repository	JPA mappings and queries	Spring test support, MySQL/test DB
Controller	HTTP contract and validation	MockMvc or equivalent
Integration	One service plus DB/clients	Spring Boot test configuration
End-to-end	All services and Compose network	Postman/Newman or integration test suite

30. TEST CASE CATALOG

The following test cases define the minimum validation set for the main order workflow. Expected outcomes must be adjusted to match the final implementation's actual status codes and response DTOs.

Test ID	Scenario	Expected behavior	Status
TC-01	GET existing customer	Returns customer DTO	Pending verification
TC-02	GET unknown customer	Returns not-found error	Pending verification
TC-03	Reduce stock by valid quantity	Stock decreases by requested amount	Pending verification
TC-04	Reduce stock by zero / negative quantity	Validation error; stock unchanged	Pending verification
TC-05	Reduce stock beyond available quantity	Conflict / domain failure; stock unchanged	Pending verification
TC-06	Place order with valid customer and stock	Order persisted; response contains identifier	Pending verification
TC-07	Place order with invalid customer	No stock mutation or order confirmation	Pending verification
TC-08	Place order with invalid product	Clear error; no successful order	Pending verification
TC-09	Order DB write fails after stock change	Compensation or documented pending state	Pending verification
TC-10	Notification service unavailable	Order and notification outcome handled by policy	Pending verification
TC-11	Repeat same order request	No duplicate side effect when idempotency is supported	Pending verification
TC-12	Container restart	Services recover and health is observable	Pending verification

Evidence to capture

For each case record request URL, method, headers, body, response status, response body, relevant database state, and service logs. Remove passwords, tokens, and personal data from screenshots. For negative inventory tests, capture the stock value before and after the request to show that the invariant was preserved.

Test status: The table intentionally marks cases as pending because startup logs alone do not establish functional test outcomes. Update each status only after executing and recording evidence.

31. OBSERVED RESULTS AND VERIFICATION STATUS

The available Docker Compose log excerpt records successful container image builds for the four application services. The stack created a Compose network and MySQL containers. Later log lines show Hikari connection pools completing initialization and Tomcat starting on the configured service ports.

Observed evidence from 13 October 2026

Check	Observed result	Interpretation
Image build	Four application images reported as built	Packaging succeeded in the captured run
Customer Service	Tomcat started on port 8081; application started	Process initialized
Product Service	Tomcat started on port 8082; application started	Process initialized
Order Service	Tomcat started on port 8083; application started	Process initialized
Notification Service	Tomcat started on port 8084; application started	Process initialized
Database pool	Hikari pool completed for each service in logs	Each service opened a DB connection
Automated tests	Build command used -DskipTests	Test results not established
End-to-end order	No successful API trace in supplied logs	Must be tested separately

Conclusion from available evidence

The runtime startup milestone is achieved in the captured run: the services initialized and their database pools connected. This is not yet evidence that the business workflow is correct, all API contracts are usable, stock updates are concurrency-safe, or notifications are delivered.

How to complete this section

Execute the test cases from page 38, collect actual HTTP responses and database-state checks, then replace “pending verification” with a real result and an evidence reference. If any behavior is not implemented, list it as a limitation rather than marking it passed.

Evidence discipline: Do not state “all tests passed” unless test reports or equivalent execution evidence support the claim.

32. TROUBLESHOOTING AND OPERATIONS RUNBOOK

Inspect container status

Common issues

Compose file path errors occur when -f is provided without a following file path. Use a quoted relative or absolute path and run docker compose config before starting. Port conflicts occur when another process already binds a host port; change the host-side mapping or stop the conflicting process. A service that cannot resolve another service may be using localhost instead of the Compose DNS name.

Database connection failures often mean a hostname, database name, username, password, or readiness condition is incorrect. Verify that service configuration uses the database container name on the internal port, and inspect MySQL logs before changing application code. JPA startup does not guarantee that every repository query or write behaves correctly.

If a service appears to run indefinitely after the “Started … Application” message, that is normal for a server process in foreground mode. Press d only when the terminal UI explicitly offers detach; otherwise Ctrl+C can stop the attached Compose process. To run the stack in the background, use docker compose up -d --build.

Data safety

Use docker compose down to stop containers. Do not remove volumes casually, because database files may be stored there. Before resetting the environment, inspect the Compose volume definitions and ensure test data can be recreated.

```
docker compose -f ".\microservices-order-system\docker-compose.yml" ps


docker compose -f ".\microservices-order-system\docker-compose.yml" logs --tail=100 order-service
```

Operational habit: After any configuration change, rebuild the affected image, inspect the container state, and re-run the API test that previously failed.

33. API CLIENT / FRONTEND INTEGRATION

A browser-based frontend or API client can provide a convenient way to exercise the order workflow. The available project context includes a frontend folder where npm install was run successfully, but the supplied evidence does not establish the frontend's implemented screens, API wiring, or end-to-end behavior. This section therefore describes the integration contract rather than claiming that a working user interface is complete.

Potential customer flow

- Load products from a product listing endpoint if that endpoint exists.
- Collect a customer identifier and one or more product quantities.
- Send an order request to the configured Order Service URL.
- Display order identifier and status returned by the API.
- Display clear validation and downstream-service errors.
- Show notification status separately from order creation status if the API exposes it.

Environment and CORS

Configure the frontend base URL through environment-specific settings rather than hard-coding service URLs in components. A browser call directly to each microservice may require CORS configuration, while an API gateway can provide one origin and a stable entry point if it is actually implemented. Do not expose database ports to the browser.

User experience and validation

Client-side validation improves usability but server-side validation remains authoritative. Disable duplicate submission while a request is pending, present loading and error states, and do not treat a client timeout as proof that the server did not process the order. Idempotency keys are useful when safe retry behavior is required.

Evidence required

Before documenting the frontend as a completed module, capture the available screens, the API requests sent by the interface, successful and unsuccessful user flows, and the command used to build the frontend.

Scope note: This section can be shortened or expanded after the frontend implementation is inspected.

34. BENEFITS, LIMITATIONS AND RISK ANALYSIS

Benefits

The project separates four business responsibilities, gives each service a clear data owner, and provides a practical environment for learning REST integration and container-based deployment. The system's boundaries make it easier to test a service independently and to discuss failure handling in a realistic backend scenario.

Current limitations to verify

Risk / limitation	Possible effect	Mitigation or next action
Synchronous dependency chain	A slow or down dependency delays order placement	Timeouts, monitoring, graceful failure
Distributed consistency	Stock may change while order persistence fails	Reservation/saga/compensation
Duplicate requests	Repeated calls can duplicate orders or reduce stock twice	Idempotency key and uniqueness constraints
Notification dependency	Notification failure may affect response	Durable event/outbox and retry state
Test evidence absent	Behavior may be assumed but unproven	JUnit, Mockito and end-to-end tests
Secrets in configuration	Credentials may leak	Environment variables and secret management
No production observability	Harder diagnosis at higher load	Metrics, trace IDs, dashboards

Risk ownership

For a learning project, documenting limitations is a positive engineering practice. Every known risk should have an owner or next action, and high-impact items should be tested before the system is described as production-ready. A successful local startup should be reported as a startup result, not generalized into a claim of availability, reliability, or scalability under load.

35. FUTURE ENHANCEMENTS

The current architecture creates a foundation that can be extended gradually. Future work should prioritize correctness and observable behavior before adding more infrastructure or extra services.

Priority	Enhancement	Value
1	Automated unit and integration tests	Proves the core service rules and contracts
2	Order idempotency	Makes client retries safer
3	Inventory reservation and compensation	Reduces risk of stock/order inconsistency
4	Transactional outbox for order events	Prevents losing events after commit
5	Asynchronous notifications	Separates message delivery from order acceptance
6	Centralized log correlation and metrics	Improves diagnosis across services
7	OpenAPI specification and API examples	Makes contracts easier to discover
8	Authentication and role-based authorization	Protects business data and operations
9	CI pipeline	Builds and tests on every change
10	Production deployment platform	Adds managed secrets, scaling and observability

Suggested implementation order

First automate current API behavior and failure paths. Next add idempotency and reliable inventory handling. Then decouple notification delivery through a persistent event or outbox pattern. Finally add security, metrics, API documentation, and a deployment pipeline. This order keeps the project centered on data correctness before infrastructure expansion.

Possible later features

Depending on the desired scope, future versions could add customer registration, product search, order history, order cancellation, payment authorization, audit trails, and an administrative view. These features should be introduced only after their business rules and service ownership are defined.

Roadmap principle: Add one capability at a time and attach test evidence to every meaningful behavior change.

36. CONCLUSION AND VIVA PREPARATION

Conclusion

The Microservices-Based Order Processing System models a business workflow through four independently deployable Spring Boot services. Customer Service, Product Service, Order Service, and Notification Service communicate through REST endpoints and use separated MySQL databases. Docker Compose makes the local environment repeatable and allows the services and databases to be inspected through container logs.

The available runtime logs confirm that all four services started and that their database connection pools initialized. The container build used a Maven packaging command that skipped tests, and the supplied evidence does not include a complete successful order API trace. Accordingly, the next milestone is functional verification with request/response evidence, database checks, automated tests, and explicit failure-path scenarios.

Viva / interview questions

- Why choose microservices instead of a monolith for this domain?
- Why should each service own its database?
- How does Order Service validate a customer and reduce product stock?
- Why is a local @Transactional annotation insufficient for two separate databases?
- What happens if stock is reduced but order persistence fails?
- How should duplicate POST /api/orders requests be handled?
- Why should notification delivery be decoupled from order creation?
- What is the difference between Docker image build success and test success?
- Why should sibling containers use Compose DNS names instead of localhost?
- How would JUnit 5 and Mockito test failure paths?
- What evidence is required before claiming the workflow passed end-to-end?

Final submission check: Update the report with actual screenshots, completed test output, repository-specific package names, and guide details before submitting.

APPENDICES AND REFERENCES

Appendix A — Final demonstration checklist

- Run docker compose config and resolve all warnings that affect correctness.
- Start the stack using the documented Compose file.
- Confirm all four application containers and MySQL containers are healthy/running.
- Show one successful customer lookup and one not-found case.
- Show stock reduction for valid quantity and rejection for insufficient quantity.
- Place a valid order and inspect its record in order_db.
- Show how Notification Service responds and how notification status is stored.
- Demonstrate one downstream failure and verify the API error response.
- Run Maven tests and retain the generated reports.
- Remove secrets and personal data from screenshots before attaching them.

Appendix B — Useful commands

Use the exact Compose filename and working directory from the final repository. Commands below illustrate common operations; adapt paths for the local project.

```
docker compose -f ".\microservices-order-system\docker-compose.yml" config
docker compose -f ".\microservices-order-system\docker-compose.yml" up --build
docker compose -f ".\microservices-order-system\docker-compose.yml" ps
docker compose -f ".\microservices-order-system\docker-compose.yml" logs --tail=100 customer-service
docker compose -f ".\microservices-order-system\docker-compose.yml" logs --tail=100 product-service
docker compose -f ".\microservices-order-system\docker-compose.yml" logs --tail=100 order-service
docker compose -f ".\microservices-order-system\docker-compose.yml" logs --tail=100 notification-service
docker compose -f ".\microservices-order-system\docker-compose.yml" down
```

Appendix C — References

1. Spring Boot Reference Documentation: https://docs.spring.io/spring-boot/

2. Spring Data JPA Reference Documentation: https://docs.spring.io/spring-data/jpa/reference/

3. MySQL 8.0 Reference Manual: https://dev.mysql.com/doc/refman/8.0/en/

4. Docker Compose Documentation: https://docs.docker.com/compose/

5. Dockerfile reference: https://docs.docker.com/reference/dockerfile/

6. JUnit 5 User Guide: https://junit.org/junit5/docs/current/user-guide/

7. Mockito documentation: https://javadoc.io/doc/org.mockito/mockito-core/latest/org/mockito/Mockito.html

8. Maven documentation: https://maven.apache.org/guides/

9. Java Platform documentation: https://docs.oracle.com/en/java/

References are listed as official learning resources. Verify versions against the project's pom.xml and Dockerfiles. Accessed or version-specific details should be updated to match the final implementation and submission date.

End of report: This document is a structured project report draft. Repository-specific details, test results, approvals, and screenshots must be reviewed and completed before formal submission.